AeroMorse — Reference

Groups, codes, the mouse, macros and secrets

This document explains how AeroMorse is used once it is built: how a dot and a dash are read, what each of the ten groups does, how the mouse works, what the screen shows, and how to change the codes and store passwords.

The per-group tables here are written by hand for the supplied `morse_map.py`. If you have changed your map, your cheat sheet and Help window show your own codes; these tables do not.

- **To look up a code**, the quickest places are the printed `AeroMorse Cheat Sheet.pdf` and the `AeroMorse Help.html` window — both are made from your own `morse_map.py`.
- **To build a device**, see the Build Guide.
- **To change a device safely**, see the Usage Guide.
- **Every setting** in `config.py` is explained in the Build Guide, Appendix E.

Contents

1. How Input Works
2. TFT Display — what the screen shows
3. Groups — every group and its codes
4. REPL Output — the device's log
5. Customising the Code Tables — macros, passwords and secrets
6. More media, browser and app-launcher keys — keys beyond Group 5, and how to add one

---

How Input Works

Dot and dash

Morse code has just two symbols, a **dot** and a **dash**. Every letter, key and command is a short run of them — a **code**, which the tables in this document list under "Pattern". The letter W is dot, dash, dash, written `.--`.

You make the two symbols like this:

You are using	To make a dot	To make a dash
Sip-and-puff	**Sip** — a short breath in	**Puff** — a short breath out
Two switches	Press the dot switch	Press the dash switch
One switch	A short press	A longer press

This document says "sip" and "puff" from here on. With switches, read "press the dot switch" for sip and "press the dash switch" for puff.

(Morse operators call the two symbols *dit* and *dah*. They mean the same as dot and dash. The documents all say dot and dash; you will only meet dit and dah if you read the firmware's own code.)

"Don't you get out of breath?"

No — because it isn't breathing. The tube is closed at the far end, where it meets the sensor, so no air travels through it. Nothing is drawn into your lungs and nothing is blown out.

A sip or a puff is a small change of pressure made with the mouth: the cheeks and tongue, the way you would start to sip a drink through a straw, or hold back a puff of air behind your lips. Your breathing carries on as usual while you do it. Jim is dependent on a ventilator to breathe and works his AeroMorse all day.

And it takes very little. As supplied, the sensor responds to a change of about 2 hPa — the pull it takes to raise a drink about 2 cm, less than an inch, up a straw. Most people use far more effort than is needed at first, and find it gets lighter and quicker with practice.

How a code is read

1. **While you sip, the device is reading a dot.** When you stop, the dot is recorded. A puff records a dash the same way.
2. **Between symbols you rest for a moment**, with no sip or puff. The dots and dashes you have entered so far show on the screen.
3. **A slightly longer rest ends the code.** After about a third of a second with no sip or puff, the device takes what you entered as one complete code, looks it up in the active group, and types that key or carries out that command.

If the code is not one the active group knows, nothing is typed and the screen shows `?` with the code you entered.

How hard a sip or puff must be, and how long the rest is, are settings in `config.py`: `THRESH_SIP` and `THRESH_PUFF` (2 hPa as supplied) and `ACCEPT_DELAY` (0.3 seconds as supplied). In the Mouse group the rest is shorter, `MOUSE_ACCEPT_DELAY` (0.15 seconds), so clicks happen sooner. The Build Guide's Appendix E explains each.

Long Press — Group Cycling

Holding a sip or puff for the `LONG_PRESS` duration cycles through groups 1–9 instead of recording a dot or dash (Group 0 is skipped — its 8-symbol patterns remain available in the background at all times):

Long press	Effect
Long sip	Cycle groups **backward** (… 3 → 2 → 1 → 8 → 7 …)
Long puff	Cycle groups **forward** (1 → 2 → … → 7 → 8 → 1 …)

The **Switch group (Group 9) is skipped** when cycling: long presses don't change group inside Switch mode (games need long holds), so cycling into it would leave you stuck. Enter Switch mode on purpose with `. - - - - - - -`.

With ten groups, cycling all the way around is slow — use the 8-symbol **Group 0 jump codes** below to go straight to any group from anywhere.

TFT Display

The 240 × 135 px display shows four text rows and a pressure bar:

Row	Content	Colour
1	Active group name, e.g. `[ Keyboard ]`	Group colour (blue / green / orange / grey)
2	Morse pattern being entered, e.g. `. - . .`	Cyan
3	Last action fired, e.g. `"hello"` or `mmove 0 -1 0`	Yellow
4	Status: armed modifiers, or SLOW / FAST / RPT / DRAG	Orange
Bar	Pressure level — green for puff, red for sip	Green / Red

At start-up row 1 shows the device's name (`DEVICE_NAME` in `config.py`, e.g. `AeroMorse Green`) in white, row 3 the firmware version (e.g. `v1.11`) and row 4 the CircuitPython version (e.g. `CP 9.2.9`), so you can see at a glance which device it is and whether it's up to date. A wireless display mirrors it. The press that closes the start-up screen is **only** a dismiss — it types nothing and doesn't change group. The Mouse-group `version` command (`. . . -`) brings the screen back. The first sip, puff or switch press switches to the normal display above.

> **No screen on your board?** Set `USE_DISPLAY = False` in `config.py`. The device then skips this local display but keeps typing over USB and keeps broadcasting these same four rows to a wireless receiver (see `USE_WIRELESS_DISPLAY`). A missing screen is auto-detected too, so a screenless board won't crash even if the flag is left on. Handy for a screenless ESP-NOW *sender* that mirrors to a #5691 *receiver*.

Groups

The device has ten groups: g0 (always-on) plus g1–g9. The active group determines which code table is used for pattern lookup. **Group 0 is always checked first**, regardless of the active group — its 8-symbol patterns are available at all times.

Group 0 — Always Available (Group Switch)

These 8-symbol patterns work in any group and jump directly to the named group. The codes use a "count of trailing dashes" scheme: 8 dots = g1, then add trailing dashes to reach the higher groups.

Pattern	Trailing dashes	Destination
`. . . . . . . .`	0	Group 1 — Keyboard
`. . . . . . . -`	1	Group 4 — Scanning / Switch Control
`. . . . . . - -`	2	Group 5 — Media (USB HID Consumer Controls)
`. . . . . - - -`	3	Group 6 — placeholder
`. . . . - - - -`	4	Group 3 — Macros
`. . . - - - - -`	5	Group 7 — placeholder
`. . - - - - - -`	6	Group 8 — placeholder
`. - - - - - - -`	7	Group 9 — **Switch** (sip holds Enter, puff holds Space)
`- - - - - - - -`	8	Group 2 — Mouse / Shortcuts
`- - - - . . . .`	(alias)	Group 4 — Scanning / Switch Control (second shortcut)

Group 0 also holds two hotkey codes for the `AeroMorse.ahk` script on the computer (Build Guide Appendix G). Being in Group 0, they work from every group:

Pattern	Sends	Does
`. - . - . -`	Ctrl+Alt+D	AeroMorse Display window on / off
`. . - . . -`	Ctrl+Alt+H	AeroMorse Help window on / off

Group 1 — Keyboard

The default group after power-on. Provides letters, numbers, punctuation, function keys, navigation keys, and modifier keys.

Two non-standard patterns free up codes for high-frequency control keys:

Letter	Standard ITU	AeroMorse	Freed code used for
M	`- -`	`- - - -`	`- -` → Backspace
C	`- . - .`	`- - - .`	`- . - .` → Left Control

Letters

Letter	Pattern	Letter	Pattern
A	`. -`	N	`- .`
B	`- . . .`	O	`- - -`
C	`- - - .` *(non-std)*	P	`. - - .`
D	`- . .`	Q	`- - . -`
E	`.`	R	`. - .`
F	`. . - .`	S	`. . .`
G	`- - .`	T	`-`
H	`. . . .`	U	`. . -`
I	`. .`	V	`. . . -`
J	`. - - -`	W	`. - -`
K	`- . -`	X	`- . . -`
L	`. - . .`	Y	`- . - -`
M	`- - - -` *(non-std)*	Z	`- - . .`

Numbers

Number	Pattern	Number	Pattern
1	`.----`	6	`-....`
2	`..---`	7	`--...`
3	`...--`	8	`---..`
4	`....-`	9	`----.`
5	`.....`	0	`-----`

Punctuation

Character	Pattern	Character	Pattern
`+`	`-...-`	`!`	`.-....`
`-`	`.----.`	`@`	`---..-`
`=`	`---.-`	`#`	`..----.`
`*`	`-..--`	`$`	`...----`
`.`	`.------`	`%`	`...-.-`
`,`	`-.....`	`^`	`-...--`
`:`	`.----.`	`&`	`.---..`
`;`	`-....-`	`?`	`-.----`
`)`	`...---`	`/`	`....--`
`(`	`---...`	`\`	`----..`
`]`	`-..---`	`|`	`....-.`
`[`	`.--...`	`_`	`----.-`
`}`	`--..-`	`"`	`...--.`
`{`	`..--.`	`'`	`..-...`
`<`	`--...--`	`` ` ``	`--.---`
`>`	`..--..`	`~`	`---.--`

Function Keys

7-symbol patterns. The dash/dot boundary shifts one step per key:

Key	Pattern	Key	Pattern
F1	`- - . - - - -`	F7	`- - - - . . .`
F2	`- - . . - - -`	F8	`- - - - - . .`
F3	`- - . . . - -`	F9	`- - - - - - .`
F4	`- - . . . . -`	F10	`- - - - - - -`
F5	`- - . . . . .`	F11	`. - - - - - -`
F6	`- - - . . . .`	F12	`. . - - - - -`

Navigation & Editing Keys

Key	Pattern	Code
Up Arrow	`.-...-`	`au`
Down Arrow	`.--...`	`ad`
Left Arrow	`.-.-..`	`al`
Right Arrow	`.-.-.`	`ar`
Home	`.......` (7 dots)	
End	`...-...`	
Page Up	`.....-`	`su`
Page Down	`...-..`	`sd`
Enter	`.-.-`	
Escape	`--....`	
Delete	`-.--..`	`kd`
Insert	`-.-...`	`ki`
Backspace	`--`	
Space	`..--`	
Tab	`---..-.`	`of`

Modifier Keys (Sticky)

Modifier keys are **sticky**: press the pattern once to **arm** the modifier. The modifier symbol will appear on the TFT display. The next key pressed fires with that modifier held, then the modifier automatically releases. Press the modifier pattern again while it is armed to **disarm** it without firing.

Shift + arrows highlights text (v1.30+). After Shift, the arrow keys — and Home, End, Page Up and Page Down — keep Shift on instead of letting it go, so Shift, arrow, arrow, arrow … keeps extending the highlight. Shift, arrow, `repeat` extends it continuously. Arm Ctrl as well as Shift to select a word at a time. Shift lets go by itself at the next key that is not one of those — Ctrl (so Ctrl then C is a plain copy), Delete, a letter — or when you send Shift again. Set `SHIFT_SELECT = False` in `config.py` for the old behaviour, where Shift let go after a single key.

Modifier	Pattern
Left Control	`-.-.`
Left Shift	`--...-`
Left Alt	`--.--`
Left GUI (Win/Cmd)	`.--.--`
Caps Lock	`------.`
Scroll Lock	`--.-..`
Num Lock	`---...-`
Print Screen	`--.---.`

Group Switch

Action	Pattern
Switch to Group 2 (Mouse)	`...-.`

Group 2 — Mouse & Shortcuts

Mouse movements follow the **numeric keypad layout**: the nine directions (up-left through down-right) map to the same positions as numpad 7–9, 4–6, 1–3. Each cardinal direction has a short pattern (2–3 symbols) and a large-step pattern (5 symbols).

Mouse Movement

Direction	Short pattern	Large pattern
↑ Up	`--`	`---..`
↓ Down	`---`	`..---`
← Left	`..`	`....-`
→ Right	`...`	`-....`
↖ Up-Left	—	`--...`
↗ Up-Right	—	`----.`
↙ Down-Left	—	`.----`
↘ Down-Right	—	`...--`
Scroll ↑	—	`.....-`
Scroll ↓	—	`...-..`

Mouse speed is controlled by three modes:

Mode	Speed multiplier	How to activate
Normal	×2	Default; also what you return to when `mslow` or `mfast` is sent a second time
Slow	×1	`mslow`, pattern `--..` (send again for Normal)
Fast	×3	`mfast`, pattern `--.-` (send again for Normal)

The effective pixels moved per step = **raw direction value × speed × 2**.

Mouse Buttons

Action	Pattern
Left click	`.-`
Right click	`.--`
Double-click left	`..-`
Double-click right	`..--`
Drag: hold the left button / let it go	`-.` (see *Drag* below)

Repeat

Action	Pattern
Toggle repeat	`.` (numpad 5)
Toggle repeat (alternate)	`. . - . .`

Repeat re-fires the last action continuously until toggled off:

- After a **mouse move**: cursor glides smoothly using pixel-accurate time-based interpolation at 40 ms intervals.
- After a **key, combo, or text**: fires that action every 40 ms.
- After switching groups, repeat is **cleared** — you must make a mouse move before repeat will activate in Group 2.

Any new sip or puff while repeating immediately **stops** the repeat.

Drag

Dragging with a normal mouse means *press the button, move, let go*. On AeroMorse the drag pattern `- .` does the pressing and the letting go for you: send it once and the left button is **held down**; send it again and the button is **released**. Everything you do with the mouse in between happens with the button held.

How to drag something:

1. Move the pointer onto the thing you want to drag — a window's title bar, a file, a scrollbar's sliding block, the start of some text.
2. Send `- .` — the button is now held. The status row shows **DRAG**.
3. Move the pointer to where you want it to go, with the normal movement patterns. Repeat (`.`) works here too, for a long glide.
4. Send `- .` again — the button is released and the thing is dropped. **DRAG** disappears from the status row.

What it is for:

To do this	Grab here (step 1)	Then move
Move a window	its title bar	to the new place
Resize a window	its edge or corner	outward or inward
Move a file or icon	the file or icon	onto the folder or spot
Select text	just before the first character	to just after the last
Select several files	an empty spot beside them	across them (draws a box)
Scroll with a scrollbar	the sliding block	up or down
Move a slider (volume, video position)	the slider knob	along the track

Good to know:

- **Use small steps near the end.** Slow speed (`mslow` , `--..`) makes it much easier to stop exactly where you want before releasing.
- **Always finish with `-.`.** Until you do, the computer believes the button is still pressed, and anything the pointer passes over may be selected or moved. If the pointer seems to be selecting things on its own, look for **DRAG** on the status row and send `-.` .
- **Changing group does not release the button.** The drag is still on when you come back to the Mouse group.
- **A click ends the drag** (v1.30+). If you send a left click while a left-button **DRAG** is showing, the held button is simply let go and **DRAG** disappears — the same as sending `-.` . Nothing is clicked a second time, so **text you highlighted by dragging stays highlighted**. A double-click lets the button go and then clicks once. A click with the *other* button lets the held button go and is then made in full — handy for a right-click on what you just selected. (v1.28 and v1.29 let the button go and then clicked again as well, which un-highlighted the text. Before v1.28 the button was let go but **DRAG** stayed on the status row until `-.` was sent.)
- **To cancel a drag** in most Windows programs, press Escape (`--....`) before releasing, then send `-.` .
- A plain **click** is often enough and less work: clicking the empty part of a scrollbar scrolls a page, and the scroll patterns move a page without aiming at anything.

Right-button drag. The firmware also accepts `mdrag right` , which holds the right button instead (in Windows, dropping a file this way offers a Copy / Move menu). It has no pattern in the supplied map; to add one, put a line such as this in the Group 2 section of `morse_map.py` :

```
g2[3][0b110] = "mdrag right"       # --.     toggle right-button drag
```

Other Controls

Action	Pattern	Effect
`mslow`	`- - . .`	Toggle slow-step mouse speed (slow ↔ normal)
`mfast`	`- - . -`	Toggle fast mouse speed (fast ↔ normal)
Find pointer	`. . - .`	Taps Ctrl by itself. If Windows' *Show location of pointer when I press the CTRL key* is on (Control Panel → Mouse → Pointer Options), circles are drawn round the pointer. Does nothing if that option is off
Centre pointer	`. - .`	Sends Ctrl+Alt+Home, which a small AutoHotkey script turns into "put the pointer in the middle of the screen". Does nothing unless `AeroMorse.ahk` is running — see Build Guide Appendix G
`version`	`. . . -`	Show the start-up screen again — device name, AeroMorse version, CircuitPython version — until the next sip/puff
`devicereset`	`- . - . . - .`	Restart the device (same as unplug/replug). Switches to Group 1 and asks **CONFIRM RESET Y/N?** — type `y` (`- . - -`) to restart; anything else, or 30 s with no input, cancels and returns to Mouse. Nothing is typed to the computer while it asks

The three speeds themselves are set in `config.py` (`MOUSE_SPEED_NORMAL` , `MOUSE_SPEED_SLOW` , `MOUSE_SPEED_FAST`).

Arrow & Keypad Keys

Group 2 also provides arrow keys and a full numeric keypad, using the same patterns as Group 1:

Key	Pattern
Up / Down / Left / Right Arrow	same as Group 1
Keypad 0–9	— (see morse_map.py)
Keypad Enter	`. - . -`
Keypad + − × ÷ .	same patterns as their Group 1 text equivalents
Application (context menu)	`- - - - . .`

Page Scroll

A scrollbar **track** click pages or line-scrolls depending on the app; these keys page in almost any app regardless of cursor position — usually the least effort for moving through a long document.

Action	Pattern	Effect
Page Down	`- . .`	Scroll down one full page
Page Up	`- . -`	Scroll up one full page
Home	`. . . . . . .`	Jump to the top of the page or the start of the line — the same code as in the Keyboard group

Windows Shortcuts

Shortcut	Pattern	Action
`. . . . .`	Win + Tab	Task View
`- - - - -`	Alt + Tab	Switch windows
`- - . . - -`	Ctrl + Alt + Left Arrow	Release mouse capture from VM
`- - - . . - .`	Tab	Tab
`- - . . . .`	Escape	Escape
`- - . . . . .`	F5	Refresh

Modifier Keys (Right-side, Sticky)

Group 2 provides the right-hand modifier keys using the same patterns as their Group 1 left-hand equivalents.

Modifier	Pattern
Right Control	`- . - .`
Right Shift	`- - . . . -`
Right Alt	`- - . - -`
Right GUI	`. - - . - -`

Group Switch

Action	Pattern
Switch to Group 1 (Keyboard)	`. . . - .`

Group 3 — Macros

Macro patterns mirror the Group 1 alphabet so the same muscle memory that types a letter also fires a macro phrase. Strings are typed through the keyboard layout writer; the host sees ordinary keystrokes.

Edit the placeholder entries in `morse_map.py` to set your own phrases. Six letter patterns come pre-filled as examples. The personal ones pull their real value from `macro_secrets.txt` via `_secret()` (see Storing passwords and secrets safely), so a shared copy of `morse_map.py` never leaks them:

Pattern	Letter	Default macro (as shipped in `morse_map.py`)
`. -`	A	`name` — `_secret('name', 'Your Name')`
`- . . .`	B	`address` — `_secret('address', …)`
`- - - .`	C	`phone` — `_secret('phone', …)`
`.`	E	`email` — `_secret('email', …)`
`. - - .`	P	`password1` — `_secret('password1', …)`
`. - -`	W	`wifi` — `_secret('wifi', …)`
`. . -`	U	`unlock` — type your PIN to unlock PIN-locked secrets (details)
`. - . .`	L	`lock` — lock PIN-locked secrets again
All other letters (D, F–K, M–O, Q–T, V, X–Z)		`'phrase'` placeholder — fill in your own

Digits `0` – `9` type the number; `. - . -` = Enter, `- -` = Backspace.

Group 4 — Scanning (Switch Control on iOS / Android)

Group 4 maps the **12 shortest Morse patterns** to Enter, Space and F3–F12, so the least-effort codes drive the most-used scan actions — a single sip is **Enter** (Select) and a single puff is **Space** (Next). This makes AeroMorse usable as a **Switch Control** scanning input on iOS and Android, where these keys act as switch actions. (Before v1.14 the single sip / puff were F1 / F2 — re-assign Select and Next in the device's switch settings if it was set up that way.) The `. . . .` pattern is set to `h` (labelled "Home"); the remaining letters and numbers are inherited from Group 1 as a placeholder and can be customised.

Key	Pattern	Key	Pattern	Key	Pattern
Enter	`.`	F5	`-.`	F9	`.-.`
Space	`-`	F6	`--`	F10	`.--`
F3	`..`	F7	`...`	F11	`-..`
F4	`.-`	F8	`..-`	F12	`-.-`
Home (`h`)	`....`				

> **Setting it up on a device:** see the **AeroMorse Switch Control Guide** for which OS action each F-key triggers and step-by-step setup for **iOS Switch Control**, **Android Switch Access**, and **Samsung Universal Switch**.

Group 5 — Media (USB HID Consumer Controls)

Group 5 sends USB HID Consumer Control codes — media playback, volume, mute, track skip, brightness, eject, and application launch. The codes route through a dedicated `ConsumerControl` HID device (enabled in `boot.py`), so they reach the host as standard media-keyboard keys that every modern OS understands.

Media — 12 shortest patterns (1–3 symbol):

Pattern	Action	Pattern	Action
`.`	Play / Pause	`...`	Stop
`-`	Mute	`..-`	Rewind
`..`	Volume Down	`.-.`	Fast Forward
`--`	Volume Up	`.--`	Brightness +
`-.`	Previous Track	`-..`	Brightness −
`.-`	Next Track	`-.-`	Eject

Application Launch (4-symbol patterns; first-letter mnemonics):

Pattern	ITU letter	Action
`-.-.`	C	**C**alculator — `AL_CALCULATOR`
`..-.`	F	**F**ile explorer — `AL_LOCAL_MACHINE_BROWSER`
`-...`	B	**B**rowser — `AL_INTERNET_BROWSER`
`.-..`	L	mai-**L** — `AL_EMAIL_READER`

The mnemonic letters above are **standard ITU** Morse. Note that g1's own `c` is the non-standard `--.` (because `-.-.` is freed there for `LEFT_CONTROL`), so `-.-.` = C applies to the ITU letter, not to g1's mapping.

These codes work on **any** `adafruit_hid` bundle. Older bundles don't define the `AL_*` constants, so `morse_map.py` resolves each one by name and falls back to its raw usage ID from the USB HID Usage Tables, Consumer Page (`0x0C`). Those IDs are fixed by the spec, so nothing is skipped and no warning is needed.

> **System `SLEEP` and `POWER` are deliberately not mapped.** Their mnemonic patterns would be `--..` (Z-z-z) and `.---` (**P**ower), but a 4-symbol pattern is too easy to hit by accident in a group whose 1–3 symbol patterns are routine media keys — and an unintended shutdown is far more disruptive than a stray volume change. `--..` and `.---` therefore keep the placeholder letters `z` and `p`. Add them in `morse_map.py` if you want them, ideally on a longer pattern.

Whether each code actually does anything is up to the host OS — Windows generally honours all four; macOS and Linux desktops vary. Many more such keys exist — browser back / forward / refresh, search, copy / paste, zoom and others — and any of them can be added to any group with one line, e.g. `_cc('AC_BACK', 0x224)`. See More media, browser and app-launcher keys below for a table of the useful ones and what to expect from them. The remaining letters and numbers in g5 are inherited from Group 1 as a placeholder and can be customised.

Group 9 — Switch (two-button hold mode)

> **Changed in v1.24:** Switch mode moved from Group 7 to **Group 9**, so the one group without Morse is the last one. Its Group 0 code is now `.--------` (it was `...------`). To keep it on Group 7, set `SWITCH_GROUP = 7` in `config.py`.

In Group 9 AeroMorse stops decoding Morse and behaves like **two plain switches**: a **sip holds Enter** and a **puff holds Space** from the moment you start until you stop — the way switch-accessible games and apps expect (e.g. Benny's Hub: Space = move, Enter = select, and games like NARBE Kart that need a key *held*).

Hard or long sips/puffs never change group here. It returns to Keyboard **by itself after 20 s** with no sip or puff (or hold one puff for 5 s). Settings: `SWITCH_GROUP`, `SWITCH_SIP_KEY`, `SWITCH_PUFF_KEY`, `SWITCH_IDLE_EXIT_S`, `SWITCH_EXIT_PUFF_S`, `SWITCH_EXIT_GROUP` in `config.py`. Full guide: AEROMORSE_SWITCH_MODE_GUIDE.md.

Groups 6, 7, 8 — Placeholders

Groups 6, 7 and 8 carry Group 1's letters and numbers on the same codes, so the same muscle memory works while you decide what each group is for. Since v1.24 they are **written out line by line** in `morse_map.py` (before, a hidden helper copied them), so to make a code do something else you just change what is between the quotes on its line — a phrase, `_secret('mykey')`, a `Keycode`, an app launcher or a command string. Groups 4 and 5 list their remaining letters and numbers the same way. Reach each group with its Group 0 jump code (table above).

REPL Output

Every action that fires is printed to the serial console (REPL) in the format:

```
<pattern>  <action>
```

Examples:

```
. - . .   "b"
. - .     KEY 82
- -       KEY 42
- - . .   mmove 0 -1 0
. - .     ?
GROUP -> 2 (Mouse)
```

`?` means the pattern was not found in any code table. Connect a terminal (Mu editor, Thonny, PuTTY, or `screen`) at 115200 baud to see this output. **Careful:** Thonny interrupts the running program when it connects, and Ctrl+C in any terminal does the same — AeroMorse then stops typing until it is restarted (replug the USB cable). If AeroMorse is your only way to use the computer, read the warning in Build Guide Appendix N first.

Customising the Code Tables

All key assignments live in `morse_map.py`. The file uses a simple dictionary structure:

```
g1[<length>][<binary_pattern>] = <action>
```

- **length** — number of symbols (1–8)
- **binary_pattern** — bits representing the pattern, MSB first; `0` = dot, `1` = dash
- **action** — one of:
  - A string like `'a'` or `'hello world'` — typed as keystrokes
  - A `Keycode` constant — pressed and released as a hardware key
  - A tuple of `Keycode` constants — all pressed simultaneously (combo)
  - A command string — `group N`, `mmove dx dy scroll`, `mclick left N`, `mdrag left`, `repeat`, `mslow`, `mfast`, `unlock`, `lock`, `version`, `devicereset` (`mreset` was removed in v1.12 — use `devicereset`)

Which key names can I use? Every valid `Keycode` name — letters, number keys, navigation, punctuation, function keys, the keypad and the modifiers — is listed in the **Keycode Reference**, a two-page sheet to keep beside you while you edit: PDF · Markdown. It also shows how to type shifted symbols such as `:` and `{`, which have no key name of their own.

Media, browser and app-launcher keys are a different kind of key, written with `CC(...)` or `_cc(...)`; see More media, browser and app-launcher keys.

Adding a Macro (Group 3 example)

```
g3[3][0b010] = 'John Smith'   # .-.  (R pattern)
```

Storing passwords and secrets safely

Group 3 macros are perfect for passwords and logins — but a password written directly into `morse_map.py` would be exposed the moment you share that file (for help, in the repo, or in a zip of your drive).

AeroMorse keeps secrets in a **separate plain-text file that is never shared**, `macro_secrets.txt`, using a deliberately foolproof format — **one secret per line,** `key=value` (no quotes, commas, colons or braces, so an editing slip can't break anything):

1. In your work folder (see the Usage Guide for the full first-time steps), copy `macro_secrets.example.txt` to `macro_secrets.txt` and put your real values in it: `password1=my-real-password wifi=my-wifi-key`
2. In `morse_map.py`, a pattern pulls a value in by key, with a harmless fallback: `python g3[4][0b0110] = _secret('password1') # P`

Why this is safe:

- If you ever share `morse_map.py` *without* `macro_secrets.txt`, the secret macros type the placeholder text — never your real password. `macro_secrets.txt` is `.gitignore`d and is the **only** file holding your secrets in plain text.

- **A typo can't take the device down.** If one line in `macro_secrets.txt` is malformed, only *that* secret falls back to its placeholder — every other secret keeps working, and `morse_map.py` always loads. (This matters: `morse_map.py` may be your only means of computer access.) A missing or unreadable file just means every secret shows its placeholder.

Format rules:

- One secret per line: `key=value`. No quotes needed; blank lines and lines starting with `#` are ignored.
- Save as **UTF-8** (in Notepad++: Encoding → **UTF-8**, *not* "UTF-8-BOM"). A stray byte-order mark is tolerated, but plain UTF-8 is cleanest.

What the value (password) may contain:

- ✅ Almost anything, with **no escaping** — `=`, `#`, quotes `' "`, backslash `\`, and symbols like `@ ! $ % & * ( ) + / ?` all work literally. Only the *first* `=` on the line splits key from value, so a value may contain further `=` signs (`k=a=b` → value `a=b`). A `#` is only special at the very start of a line.
- ✅ Interior spaces are kept (`name=Your Name`).
- ⚠ **Leading and trailing spaces are trimmed.** `pw= x` stores `x`. (The trim is deliberate — it removes the invisible carriage-return Windows adds to each line.) So a value that must *begin or end with a space* can't be stored as-is.
- ⚠ **No newlines** — a value is a single line, so it can't span lines.

What the key (the name before `=`) may contain:

- ✅ Letters, digits, and `_` (e.g. `email_pw`, `bank_pin`, `wifi2`). The key must match the `_secret('key', …)` spelling in `morse_map.py` exactly.
- ❌ **No** `=` (the first `=` ends the key), and a key **can't start with** `#` (that line is read as a comment). Avoid spaces in keys.

Works in any group, not just Group 3. `_secret()` reads from the same `macro_secrets.txt` no matter which group calls it, so you can use `_secret('key', 'placeholder')` on any assignment in **g1–g9** — e.g. dedicate a placeholder group like g6 to logins:

```
g6[4][0b0110] = _secret('bank_login')  # P
```

- **Keys are shared across the whole file** — `_secret('email', …)` in g3 and in g6 both read the same `email=` line. Use distinct key names (`email`, `work_email`) if you want different values.
- In **g4–g8** the letters and numbers are ordinary lines in the file: to put a secret on a letter's code, replace the quoted letter on that line with `_secret('mykey')`.
- **Short form (v1.24+):** `_secret('mykey')` is enough. The second argument (the text typed when the key isn't set) is optional; without it the pattern types `(set mykey in AeroMorse Secrets)`.
- The cheat sheet shows a 🔒 lock badge with the key name for `_secret(...)` in **any** group, so secrets stay hidden when printed.

> **If you send someone a copy of your whole CIRCUITPY drive, delete `macro_secrets.txt` from the copy first.** The `.gitignore` protects git and one-file sharing; a full-drive copy still needs that one manual step.

PIN-locked secrets (`macro_secrets.enc`)

`macro_secrets.txt` is plain text: anyone who plugs the device into a computer can open it. From **v1.5** you can instead keep your secrets **encrypted**, like a password manager, and unlock them with a PIN you type in Morse.

Set it up (on your PC):

1. Double-click `AeroMorse Secrets.exe` in your work folder (or run `python aeromorse_secrets.py`). It opens the secrets in that folder — your existing `macro_secrets.txt` is imported automatically.
2. Edit the `key=value` lines (same rules as above), type a **PIN** twice, and click **Save (encrypt)**. This writes `macro_secrets.enc` and offers to delete the plain `macro_secrets.txt` (recommended). It also warns you about any `_secret()` key in `morse_map.py` that has no value.
3. Click **Copy to device**. It copies the file to the CIRCUITPY drive and offers to delete the plain `macro_secrets.txt` there too.

Use it (on the device):

- The device starts **locked** at every power-up.
- Use any secret pattern (e.g. Macro `P`) and the screen shows **ENTER PIN + ENTER**. Type the PIN with the normal **Group 1** letters/digits, then **Enter** (`. - . -`). The screen shows `PIN ****` ; **nothing you type while entering the PIN reaches the computer**.
- Right PIN → **UNLOCKED** (about 1.5 s) and it types the secret you asked for. The status line shows `UNLOCKED` while secrets are open — only in groups whose map has `_secret()` entries (v1.19+), so it doesn't clutter the other groups.
- Wrong PIN → **WRONG PIN**, stays locked; just use the pattern again.
- **Esc**, or **Backspace** with nothing typed, cancels. Entry also cancels itself after 60 s of no input.
- While the PIN is being typed you **stay in Group 1**: a strong sip/puff counts as a normal dot/dash, and long presses and group codes don't change group. (The same applies to the `devicereset` "CONFIRM RESET Y/N?" prompt.)
- Macro `U` (`. . -`) unlocks without typing anything; Macro `L` (`. - . .`) locks again. Unplugging or restarting always locks.
- Optional idle auto-lock: `SECRETS_AUTOLOCK_MIN` in `config.py` (0 = off).

Good to know:

- The PIN is **not case-sensitive**. Use **8 or more** letters/digits: the file is strongly encrypted, but a short PIN could be guessed by someone who copies the file and tries PINs on a computer.

- **Forgot the PIN?** Only the secrets are affected — keyboard, mouse and everything else keep working. Make a new file with the tool.
- Secret values are never shown on the screen, the wireless display or the USB log — only `SECRET name`. (Before v1.5 the first 16 characters were shown.) While a PIN is typed, the wireless display shows `*` instead of dots and dashes.
- If both files are on the device, the encrypted one is used and the USB log warns that the plain file should be deleted; the validator warns too.
- Needs CircuitPython's built-in `aesio` module (present on the ESP32-S3 Feathers). `morse_map.py`, `code.py`, `config.py` and `boot.py` must all be **v1.5 or later**.

Changing a Mouse Move Step

```
g2[2][0b11] = "mmove 0 -2 0"  # double the up-step
```

More media, browser and app-launcher keys

Group 5 (Media) comes with volume, play / pause, track, brightness and four app launchers. Those are **Consumer Control** keys — the same kind of key as the extra buttons on a multimedia keyboard. The USB standard defines a few hundred more, and **AeroMorse can already send any of them**; they just are not assigned to a code out of the box.

How to add one

In `morse_map.py`, in any group, use `_cc()` with a name of your choosing and the key's number:

```
g2[6][0b101011] = _cc('AC_BACK', 0x224)      # -.-.--  browser Back
```

- The **number** is what matters (`0x224` is "Back" in the USB standard).
- The **name** is only tried first, for newer key libraries that know it; if it is unknown the number is used. Any name works.
- No extra file or library is needed.

Pick a free code for it (`morse_map_report.txt` lists them), then validate and copy as for any change to the map (Usage Guide §3).

Keys worth knowing

What it does	Line to use	On screen
Browser: back	`_cc('AC_BACK', 0x224)`	BROWSER BACK
Browser: forward	`_cc('AC_FORWARD', 0x225)`	BROWSER FWD
Browser: reload the page	`_cc('AC_REFRESH', 0x227)`	BROWSER REFRESH
Browser: stop loading	`_cc('AC_STOP', 0x226)`	BROWSER STOP
Browser: home page	`_cc('AC_HOME', 0x223)`	BROWSER HOME
Browser: favourites / bookmarks	`_cc('AC_BOOKMARKS', 0x22A)`	BOOKMARKS
Search	`_cc('AC_SEARCH', 0x221)`	SEARCH
Calculator	`_cc('AL_CALCULATOR', 0x192)`	CALCULATOR
File explorer ("This PC")	`_cc('AL_LOCAL_MACHINE_BROWSER', 0x194)`	FILE EXPLORER
Web browser	`_cc('AL_INTERNET_BROWSER', 0x196)`	WEB BROWSER
Mail	`_cc('AL_EMAIL_READER', 0x18A)`	MAIL
Media player	`_cc('AL_CONSUMER_CONTROL_CONFIGURATION', 0x183)`	MEDIA PLAYER
Calendar	`_cc('AL_CALENDAR', 0x18E)`	CALENDAR
Copy / Cut / Paste	`_cc('AC_COPY', 0x21B)` / `0x21C` / `0x21D`	COPY / CUT / PASTE
Undo	`_cc('AC_UNDO', 0x21A)`	UNDO
New / Open / Save / Print	`_cc('AC_NEW', 0x201)` / `0x202` / `0x207` / `0x208`	NEW / OPEN / SAVE / PRINT
Find	`_cc('AC_FIND', 0x21F)`	FIND
Zoom in / out	`_cc('AC_ZOOM_IN', 0x22D)` / `0x22E` — **does nothing on Windows**; use `Keycode.CONTROL, Keycode.EQUALS` and `Keycode.CONTROL, Keycode.MINUS` instead	ZOOM IN / ZOOM OUT

Media keys already in Group 5 can be reused in other groups by their plain names, for example `CC(ConsumerControlCode.MUTE)` or `CC(ConsumerControlCode.VOLUME_INCREMENT)`.

A key that is not in the table shows on the screen as `CC` and its number.

The full list — about 225 named keys — is in the community file consumer_control_extended.py (Neradoc, MIT licence). You do not need to copy that file to the device; read the number you want from it and use it in `_cc()` as above.

What to expect

> **Whether a key does anything is up to the computer, not AeroMorse.** The device sends the key correctly either way; the operating system and the program in front decide whether to act on it.
>
> - **Windows** acts on the browser keys, Search, and the launchers for Calculator, File Explorer, Mail and Media player in most programs. Copy / Paste / Save / Print and the like are honoured by some programs (Microsoft Office, for one) and ignored by many others — for those, a normal key combination such as `Keycode.CONTROL, Keycode.C` is the dependable choice.
> - **macOS, Linux, phones and tablets** support fewer, mainly the media keys.
> - **Tried by the project:** the Group 5 media keys and the Calculator launcher work on Windows. **Zoom in / Zoom out were tried on Windows and did nothing** — to zoom the page use Ctrl with plus or minus (`Keycode.CONTROL, Keycode.EQUALS` / `Keycode.CONTROL, Keycode.MINUS`), or for the Windows Magnifier the Windows key with plus or minus (`Keycode.GUI, Keycode.KEYPAD_PLUS` / `Keycode.GUI, Keycode.KEYPAD_MINUS`). The rest of the table is from the USB standard and has not been tried key by key.
>
> If a key does nothing, nothing is wrong with the device — that computer simply does not use it.

Leave these alone unless you mean it: `SLEEP` (`0x32`) and `POWER` (`0x30`) put the computer to sleep or switch it off. If you add one, give it a long code that cannot be entered by accident.